

Introduction to Machine Learning (67577): Hackathon 2026

Challenge 1: Bike Demand Forecasting

June 24, 2026

1 Motivation

Bike-sharing systems are an important part of urban transportation. In many cities, rental-bike companies operate hundreds of stations and must decide how many bikes should be available at each station throughout the day. This is a challenging forecasting problem: demand depends on time of day, day of week, station location, commuting patterns, holidays, weather, city-specific behavior, and many other factors.

In this challenge, you will act as a data science team for The Chain Rule, a bike-rental company operating in several cities. Your goal is to predict hourly bike demand at each station. More precisely, for a given city, station, and hour, you should predict how many rides will start from that station during that hour.

The challenge is designed to test both predictive modeling and generalization. A good solution should perform well on the main cities observed in the data, while also being able to make reasonable predictions for a new city with only a limited amount of available history.

2 Tasks

The main evaluation focuses on `city1` and `city2` - the two major cities in the challenge. An additional competitive component is evaluated on `city3`, which represents a generalization setting with limited available data.

Your submission must contain a model trained from random weight initialization that can be loaded by the evaluator and used to predict demand for every requested station-time row. Each individual example corresponds to one city, one start station, and one specific time interval. The exact prediction target is the total number of rides that originate from that station during that time interval.

Formally, for a given city c , start station s , and time interval t , the demand label is defined as:

$$y_{c,s,t} = \#\{\text{rides starting from station } s \text{ during time interval } t\}.$$

2.1 Bi-Dataset Predictor: City 1 and City 2

The first part of the challenge is to build a predictor that performs well on the two main cities, `city1` and `city2`. These cities may differ in size, station distribution, demand patterns, missing values, and feature availability.

Your model should learn useful patterns from both cities and produce accurate station-hour demand predictions.

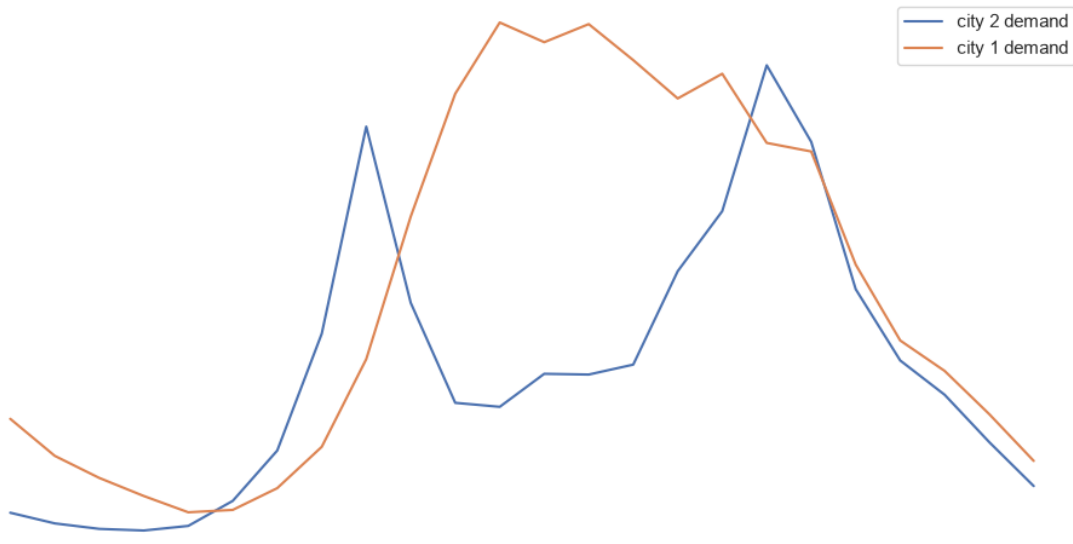


Figure 1: The two lines exhibit clearly different temporal patterns, indicating that a single global trend is insufficient. This motivates using group-specific calibration or features that allow the model to adapt to different distributions.

2.2 City 3 Generalization: Learning from a Small Portion

The second part of the challenge evaluates how well your model handles `city3`, representing a more difficult generalization setting. Only a small portion of `city3` history will be available, and your model should still be able to make reasonable predictions, and will be tested against data from a fourth, held-out city.

This part encourages robust modeling choices. A model that relies too heavily on memorizing station identifiers from the main cities may fail on `city3`. A stronger solution may combine station-level history, city-level demand patterns, calendar features, weather, station metadata, and fallback statistics that remain useful when recent history is limited.

3 Evaluation

The goal of your model is to achieve low prediction error on hourly station demand. The final evaluation will include the two main cities, and an additional smaller city.

The main error metric is mean absolute error:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

The evaluator clips negative predictions to zero before calculating the loss $\hat{y}_i^{\text{used}} = \max(0, \hat{y}_i)$, so lower MAE is better.

You can assume that the test data will be evenly distributed between tested cities.

4 Data and Setup

To get started, first download the project files from [Moodle](#). Then, download the dataset from [here](#), and place the files according to the directory tree chart below.

Starter Project Structure:

```

project/
├── dataset/
│   └── # place train_set.csv here
├── submissions/
│   ├── my_team/# Contains templates for your code. Start working here
│   │   ├── model.py
│   │   ├── train.py
│   │   └── predict.py
│   └── dummy_submission/# Baseline example scripts
│       ├── model.py
│       ├── train.py
│       └── predict.py
├── evaluate.py
├── base_model.py
├── build_station_hour_eval_data.py
└── check_submission_format.py

```

4.1 Project Files Overview

- `dataset/`: Directory where you should place the downloaded data files.
- `submissions/my_team/`: Your working directory where your specific team's code will reside.
 - `train.py`: learns parameters/artifacts from training data, and saves all required artifacts to `weights.joblib`.
 - `model.py`: Contains your model logic, helper functions, and feature engineering code.
 - `predict.py`: Defines the standard evaluator interface; **do not modify this file**. It loads `weights.joblib`, calls code from `model.py`, and returns predictions.
 - `README`: Should contain your team information and a short description of your model design.
- `dummy_submission/`: A minimal working example containing a basic `train.py`, `model.py`, `predict.py`, and dummy weights.
- `evaluate.py`: The local evaluator script that will score all submission folders under `submissions/`.
- `base_model.py`: Defines the base interface that your submitted model must inherit from.
- `build_station_hour_eval_data.py`: Converts ride-level validation data into the evaluation format (public target rows and private labels).

4.2 Raw Data Format

The raw data consists of ride-level logs from three cities. Each row corresponds to a single bike ride. The exact fields may differ between cities, and part of the challenge is to inspect, clean, and align the data.

These fields cover ride timing, station identifiers, user metadata, station location, weather conditions, city identifiers, nearby infrastructure and points of interest, calendar variables, and hourly timestamps.

Important: Test Input Features

The cities are not guaranteed to have the same number of rides, stations, or missing values. Your model should avoid assumptions that work only for one city and fail on another. During inference, your model will be given inputs in the following format:

```
started_at, ended_at, start_station_id, end_station_id, usage_time_minutes,
distance_meters, user_type, start_lat, start_lng, temperature_2m, relative_humidity_2m,
apparent_temperature, precipitation, rain, snowfall, cloud_cover, wind_speed_10m,
bike_lane_length_500m, park_area_500m, university_count_1000m, city,
office_poi_count_1000m, retail_poi_count_1000m, restaurant_cafe_count_500m,
transit_stop_count_500m, distance_to_nearest_rail_station, distance_to_city_center,
date, weekday, weekend, holiday, holiday_name, working_day, hour_ts
```

You can assume that the set of possible values for categorical features is identical between train and test.

4.3 Aggregated Demand Format

Your model should operate on station-hour examples. Each example corresponds to one city, one start station, and one hour. The prediction target is the number of rides that start from that station during that hour.

Formally, station s , date d , and hour h , the demand label is

$$y_{s,d,h} = \#\{\text{rides starting from station } s \text{ during hour } h \text{ on date } d\}.$$

For each row in a validation/test file, your model should produce one prediction for the corresponding `start_station_id`, and `hour_ts`.

5 Model Development Workflow**5.1 Local Validation Construction**

As the very first step in your development process, you must create a local validation framework to reliably measure your model's performance and simulate the official evaluation. To do this, split the original training data (`dataset/train_set.csv`) into a new training set and your own local validation set:

```
dataset/local_train_set.csv
dataset/local_validation_set.csv
```

Next, run the provided script to convert your `local_validation_set.csv` into the format used by the evaluator. This splits your validation data into an input file (the features you use to predict) and a private labels file (the ground-truth answers):

```
python build_station_hour_eval_data.py \
  --input_csv dataset/local_validation_set.csv \
  --public_targets_csv dataset/public_validation_targets.csv \
  --private_labels_csv dataset/private_labels.csv
```

From this point on, your local validation set acts as your test set. The `public_validation_targets.csv` is the input your model will predict on. The `private_labels.csv` contains the true demand values and is used purely by the evaluator script to calculate your score. **Do not train your model on your validation set**, and do not use the demand column from the `private_labels.csv` as a feature during prediction.

5.2 Recommended Modeling Pipeline

1. Split the original data into `local_train_set.csv` and `local_validation_set.csv`.
2. Run the formatting script to create your validation targets and private labels.
3. Aggregate raw training rides into hourly station demand (using only your `local_train_set.csv`).
4. To check the setup, run `dummy_baseline/train.py` and then execute the evaluation procedure described in 6.4. The output should report the MAE for the dummy baseline and include a failure comment for `my_team`, as that component has not yet been implemented.
5. Build and process features.
6. Train a model on your station-hour training examples.
7. Save all objects needed for prediction into `weights.joblib`.
8. Test your performance on your local validation set using `evaluate.py`.
9. Iterate over step 4-7 to improve performance.

Tip

Your model should support arbitrary target time slots, including cases where the requested test time is not close to the most recent observed record. For example, a test instance may occur after a gap of several days or weeks from the last available observation. In such cases, recent lag features may be unavailable or stale, so your model should be able to fall back on more stable features.

6 Submission Instructions

6.1 Prediction Format

The challenge consists of submitting a trained bike-demand forecasting model. During evaluation, your model will receive a pandas dataframe containing station-hour target rows, and should return one predicted demand value for each row.

Formally, for an input target row $x = (c, s, t)$, where c is the city, s is the start station, and t is the requested hour, your model should output:

$$\hat{y} = f_{\theta}(c, s, t), \quad \hat{y} \geq 0.$$

Your `predict` function should return an array-like object of length `len(test_df)`, where each value is the predicted number of rides starting from the requested station during the requested hour.

6.2 Submission Folder Layout

Only one team member should submit the project on Moodle. Submit a folder named explicitly matching the challenge number and your student IDs. Your team folder must be named `challenge_1_IDs` (for example: `challenge_1_123456789_234567891`).

Your internal submission folder must contain exactly these five files and follow this layout:

```
challenge_1_IDs/  
  train.py  
  model.py  
  predict.py  
  weights.joblib  
  README
```

6.3 General File Requirements

The evaluator uses `predict.py`, `model.py`, and `weights.joblib`. The evaluator does *not* run `train.py` during grading. The file `train.py` is included so that your work is completely reproducible.

- `train.py`: Must run from inside the submission folder. It should train your model from scratch and then save all fitted objects, parameters, or state dictionaries needed for prediction exactly to `weights.joblib`.
- `model.py`: Should contain the internal model code, helper functions, feature engineering logic, or model architecture used by your submission.
- `predict.py`: Must remain structurally unchanged from the provided evaluation wrapper interfaces. Do not change the function signatures in this file.
- `README`: Must contain your information and a short description of your model design. The first line should contain the team members' names, separated by commas with no spaces. The second line should contain the team members' IDs, separated by commas with no spaces. After a blank line, briefly describe your model architecture and provide instructions for reproducing your model.

Do not submit:

the dataset; virtual environments; cache files; unnecessary checkpoints; files that rely on absolute paths from your computer, such as `C:/Users/...`

6.4 How to Test Yourself Locally

Once you have trained your model and saved the outputs in your submission folder, run the local evaluator to see how well you predict against your validation set:

```
python evaluate.py \  
--eval_dir dataset \  
--submissions_dir submissions \  
--output_csv mae_by_city.csv
```

The evaluator writes `mae_by_city.csv`.

You may also want to run `check_submission_format.py`:

```
python check_submission_format.py
```

It checks each submission folder, loads the required scripts and `weights.joblib`, calls `Model.load()`, and then calls `Model.predict()` on the target dataframe representing your local validation set.

6.5 Final Checklist

Before submitting, check that:

- your submission folder contains `train.py`, `model.py`, `predict.py`, `weights.joblib`, and `README`.
- `train.py` runs from inside the submission folder and saves all fitted objects to `weights.joblib`;
- `train.py` reads the training data from `../dataset/train_set.csv`.
- `model.py` defines a class named `BikeDemandModel`;
- `BikeDemandModel` contains your model logic, preprocessing, feature engineering, and prediction code;
- `predict.py` is unchanged from the provided template;
- `predict(test_df)` returns one numeric prediction per row in `test_df`;
- `predict(test_df)` does not require a demand column;
- predictions are non-negative or clipped to zero;
- your code does not depend on the hidden test files;
- your code does not rely on absolute local paths such as `C:/Users/...`

7 Interview.

As part of the final evaluation, each team will participate in a short interview. During the interview, you should be prepared to explain your modeling choices, describe the reasoning behind your design decisions, and walk the interviewer through your implementation and results. No presentation is required; the interview is intended to assess understanding and the ability to discuss the solution clearly.